



Université paul vallery Montpellier 3

Licence Mathématique et informatique appliquées au sciences
humaines et sociales (MIASHS)

**TE38XI/TE58XI - Compétences en informatique
S3 et S5 : Spécialisation projets**

**Modélisation Multi-Agents d'un
Écosystème Proie-Prédateur : Analyse
technique et comportemental de la
simulation « sheep-wolf »**

Réalisé par :
Aina Akorede Tylden Hounsa

Encadré par :
Jérôme Pasquet

Année universitaire 2025-2026

Table des matières

.....	1
1. Contexte du Projet.....	3
2. Architecture logiciel	3
3. Analyse Technique : Pathfinding et Coordonnées Continues	4
3.1. Le Passage aux Coordonnées Continues	4
3.2. Algorithme A* Standard (world.py).....	4
3.3. Algorithme A* Memory (agent.py - Spécifique au Loup).....	5
4. Scénarios et Émergences Attendues.....	6
Scénario A : Équilibre Précaire.....	6
Scénario B : Surpopulation de Prédateurs.....	6
Scénario C : Famine des Herbivores (Carence de Ressources).....	6
5. Analyse des Résultats par rapport aux Scénarios	7
5.1. Efficacité de la Mémoire du Loup.....	7
5.2. Comportement de Fuite du Mouton	7
5.3. Gestion de l'Énergie	7
6. Semainier de réalisation.....	7
Conclusion.....	8

1. Contexte du Projet

Ce projet s'inscrit dans le cadre de la modélisation des systèmes complexes par une approche de modélisation sur la base d'agents. L'objectif est de simuler un écosystème simplifié comprenant trois entités principales : les ressources (Herbe/Eau), les proies (Moutons) et les prédateurs (Loups).

Le système repose sur une architecture distribuée :

1. **Le Cœur de Simulation (Python)** : Gère la logique des agents, les threads concurrents, la physique du monde et le pathfinding.
2. **La Visualisation (Java/JavaFX)** : Affiche l'état du monde en temps réel via une communication socket TCP/IP.

Contrairement à des modèles mathématiques globaux (type équations de Lotka-Volterra), ce projet mise sur l'émergence de comportements globaux à partir de règles individuelles simples (déplacement, faim, champ de vision, mémoire).

2. Architecture logiciel

L'application repose sur un modèle Client-Serveur asynchrone, assurant une séparation stricte entre le moteur de calcul (Python) et l'interface de visualisation (Java). Cette architecture modulaire facilite la maintenance et l'évolutivité du projet.

2.1. Le Backend (Python) : Moteur de Simulation

Le serveur est structuré autour de plusieurs modules interconnectés, gérant la complexité du parallélisme :

- **world.py (L'Environnement Thread-Safe)** : C'est le conteneur de l'état global. Il utilise un **verrou ré-entrant (threading.RLock)** crucial pour synchroniser l'accès aux dictionnaires d'agents et de ressources. Cela prévient les "Race Conditions" lorsqu'un agent modifie l'état du monde (ex: manger une ressource) pendant que le thread principal lit l'état pour l'envoyer au client.
- **agent.py (Intelligences Autonomes)** : Chaque agent (Sheep ou Wolf) est un **thread indépendant** (threading.Thread). Ils possèdent leur propre boucle de vie (run), gérant leur cycle Perception-Décision-Action (Machine à états finis implicite : Errance, Chasse, Fuite).
- **main.py (Orchestrateur & Réseau)** : Il initialise la simulation, lance les threads, et gère la boucle principale d'envoi de données. Il agit comme un serveur TCP (socket.AF_INET), sérialisant l'état du monde en JSON à une fréquence régulée (send_frame).

2.2. Le Frontend (Java) : Visualisation Déportée

Le client est une application **JavaFX** passive qui consomme le flux de données :

- **Gestion de la Concurrency UI** : Un thread d'écoute dédié lit le flux TCP en continu pour ne pas bloquer l'interface graphique.
- **Mise à jour Graphique** : L'utilisation de `Platform.runLater()` est impérative pour manipuler le graphe de scène (Scene Graph) depuis le thread réseau, garantissant la thread-safety de JavaFX.
- **Rendu Vectoriel** : Les agents sont représentés par des objets `Circle` et les ressources par des `Rectangle`, dont les coordonnées sont mises à l'échelle dynamiquement selon la résolution de la fenêtre.

2.3. Protocole de Communication

La communication est unidirectionnelle (Serveur vers Client) et repose sur l'échange de **snapshots JSON** délimités par des sauts de ligne. Chaque paquet contient l'intégralité des positions et états (type, id), permettant au client de redessiner complètement la scène à chaque frame (Stateless rendering).

3. Analyse Technique : Pathfinding et Coordonnées Continues

Une des branches les plus critiques du code réside dans la gestion des déplacements. Le défi technique majeur a été de concilier un algorithme de recherche de chemin discret (sur une grille) avec des déplacements d'agents continus et fluides.

3.1. Le Passage aux Coordonnées Continues

Bien que le monde soit conceptuellement une grille de 50x50 évolutive (`parametres.py`), les agents possèdent des coordonnées flottantes (`self.x`, `self.y`).

Dans la méthode `run()` des agents (`agent.py`), le déplacement n'est pas une simple téléportation de case en case, mais interpolé :

- Le chemin est calculé comme une liste de nœuds entiers.
- L'agent calcule un vecteur de direction normalisé vers le prochain nœud cible.
- À chaque itération de la boucle (tick), l'agent avance d'un `step` proportionnel à sa vitesse (`current_speed`) et au temps de pause CPU.

Extrait de logique (pseudocode) :

```
1. dx = noeud_suivant_x - self.x
2. dy = noeud_suivant_y - self.y
3. dist = math.hypot(dx, dy)
4. # Avancée partielle fluide
5. self.x += (dx / dist) * step
6. self.y += (dy / dist) * step
```

Cela permet une simulation visuellement fluide côté client Java et une gestion plus fine des collisions (rayon de 1.0 pour manger/tuer).

3.2. Algorithme A* Standard (`world.py`)

Cette version de l'algorithme est implémentée dans la méthode `world.astar`. Elle permet de trouver le chemin le plus court entre deux points en tenant compte des obstacles (l'eau). Voici le détail de son fonctionnement interne :

A. Discrétisation et Initialisation

L'algorithme reçoit des coordonnées flottantes (position de l'agent et de la cible) et commence par les convertir en coordonnées entières (nœuds de la grille) :

- `start_node = (int(start[0]), int(start[1]))`
- `goal_node = (int(goal[0]), int(goal[1]))`

Il initialise ensuite une file de priorité (heapq) nommée `open_set` pour explorer les nœuds les plus prometteurs en premier.

B. La Fonction de Coût ($f = g + h$)

Pour chaque nœud, l'algorithme calcule un score **f** qui détermine l'ordre d'exploration :

- **g(n) (Coût réel)** : La distance parcourue depuis le départ jusqu'au nœud actuel.
- **h(n) (Heuristique)** : Une estimation du coût restant pour atteindre la cible.
 - *Implémentation choisie* : L'heuristique utilisée est la **distance Euclidienne** (`math.hypot`), ce qui est cohérent car les agents peuvent se déplacer en diagonale.
 - Formule : $\text{sqrt}(x_2 - x_1)^2 + (y_2 - y_1)^2$

C. Exploration des Voisins (8-Connexité)

Contrairement à des grilles simples (4 directions : Nord, Sud, Est, Ouest), l'algorithme explore **8 directions** (incluant les diagonales : (-1,-1), (1,1), etc.). Pour chaque voisin potentiel :

1. **Vérification des limites** : Le voisin doit être dans les bornes du monde ($0 \leq x < \text{width}$).
2. **Vérification des obstacles** : La méthode appelle `self.is_cell_traversable(x, y)` qui vérifie la présence d'eau via `self.has_water_at`. Si de l'eau est présente, le nœud est ignoré.

D. Reconstruction du Chemin

Lorsque le nœud courant correspond au `goal_node`, l'algorithme remonte la chaîne des parents stockés dans le dictionnaire `came_from` pour reconstruire le chemin inverse, puis le remet dans l'ordre (du départ vers l'arrivée).

Si aucun chemin n'est trouvé (cible inaccessible entourée d'eau), la fonction retourne une liste vide [].

3.3. Algorithme A* Memory (`agent.py` - Spécifique au Loup)

Cet algorithme fonctionne quasiment de la même manière que le A* standard a quelques exceptions près :

A. Le Filtre de Mémoire ("Mur Invisible")

Avant même de lancer la recherche, l'algorithme vérifie si la destination finale est connue.

- **Problème** : Si un mouton est visible mais court vers une zone non explorée, un A* standard échouerait car le nœud d'arrivée est considéré comme inaccessible.
- **Solution** : Le loup scanne sa mémoire (`known_map`) pour trouver la case connue la plus proche de la proie réelle. Il se dirige alors vers cette "frontière" de son savoir, espérant découvrir le reste du chemin une fois arrivé sur place.

B. Le Filtre de Mémoire ("Mur Invisible")

Lors de l'exploration des voisins dans la boucle principale, une condition supplémentaire est appliquée : `if not self.is_known(neighbor[0], neighbor[1]): continue`. Une case parfaitement praticable (herbe) est traitée comme un obstacle infranchissable tant qu'elle n'a pas été visitée ou scannée par le champ de vision. Cela force l'agent à contourner les zones d'ombre plutôt que de couper à travers l'inconnu..

4. Scénarios et Émergences Attendues

Nous avons défini trois scénarios basés sur les paramètres initiaux (`parametres.py` et `main.py`) pour observer les dynamiques de population.

Scénario A : Équilibre Précaire

- **Configuration** : 20 Moutons, 4 Loups, Ressources abondantes.
- **Déroulement** : Les moutons ont assez d'espace pour fuir. Les loups, peu nombreux, doivent explorer (mémoire vide au départ).
- **Émergence attendue** : On observe des phases de "course-poursuite". Les moutons forment parfois des groupes compacts implicites en fuyant une menace commune. Les loups survivent en éliminant les moutons faibles ou isolés. C'est le scénario le plus "durable".

Scénario B : Surpopulation de Prédateurs

- **Configuration** : 15 Moutons, 15 Loups.
- **Déroulement** : La pression de prédation est immédiate. Le rayon de vision des loups couvre la majorité de la carte rapidement.
- **Émergence attendue** : "Massacre rapide". Les moutons sont éliminés en quelques secondes. Ensuite, les loups, ayant consommé toute la biomasse disponible, meurent de faim collectivement une fois leur énergie épuisée (`energy <= 0`). Ce scénario démontre la fragilité des chaînes alimentaires.

Scénario C : Famine des Herbivores (Carence de Ressources)

- **Configuration** : 30 Moutons, 2 Loups, très peu
- **Déroulement** : Les moutons consomment l'herbe toute l'herbe disponible
- **Émergence attendue** : Compétition intra-spécifique. Les moutons ne meurent pas des loups, mais de leur propre nombre. On observe des moutons errer à la recherche de cibles inexistantes jusqu'à l'épuisement énergétique.

Évaluation : Le **Scénario A** est le plus approprié pour la démonstration finale. Il permet de mettre en valeur l'intelligence artificielle (IA) des agents (fuite et chasse) sur la durée, tandis que les scénarios B et C mènent à une fin de simulation trop abrupte.

5. Analyse des Résultats par rapport aux Scénarios

L'exécution de la simulation confirme plusieurs hypothèses comportementales codées dans `Sheep` et `Wolf`.

5.1. Efficacité de la Mémoire du Loup

Dans les débuts de la simulation (quel que soit le scénario), les loups sont moins efficaces. L'analyse des logs montre qu'ils passent beaucoup de temps en mode "Exploration" (`random known position`). Ce n'est qu'après avoir quadrillé la zone que la fonction `astar_memory` devient redoutable, car elle permet de couper la trajectoire des moutons via des raccourcis connus. Cela valide l'implémentation de la classe `Wolf`.

5.2. Comportement de Fuite du Mouton

Le mécanisme de fuite (`angle = math.atan2...`) fonctionne bien pour la survie immédiate. Cependant, dans le **Scénario B**, on remarque que les moutons se "coincent" souvent dans les coins de la carte (limites 0 et `world.width`).

- **Observation** : L'algorithme de fuite simple les pousse à l'opposé du loup, parfois directement vers un cul-de-sac ou un point d'eau non traversable, facilitant la capture. Une amélioration serait d'intégrer les obstacles dans le vecteur de fuite.

5.3. Gestion de l'Énergie

Le paramètre `hungry_threshold` (80% et 66%) joue un rôle crucial.

- Si ce seuil est trop bas, l'agent attend d'être critique pour chercher à manger, et meurt souvent en chemin (famine).
- Avec le réglage actuel, l'alternance entre les états "Promenade" et "Recherche de nourriture" crée un cycle de vie dynamique. Cependant, l'absence de reproduction dans le code actuel fait que la courbe de population est strictement décroissante.

6. Semainier de réalisation

Ce planning retrace l'évolution incrémentale du projet, de la conception architecturale à l'optimisation des algorithmes complexes.

- **Phase 1 : Fondations et Architecture (Semaines 1 à 3)**
 - Mise en place de l'environnement de développement et du dépôt Git.
 - Conception UML et implémentation des classes socles : World (Singleton thread-safe) et Agent (Thread).
 - Intégration du module `ressources.py` pour la gestion statique de l'environnement.
- **Phase 2 : Physique et Mouvement (Semaines 4 à 6)**
 - **Défi technique :** Transition d'un modèle discret (case par case) à un modèle continu (coordonnées flottantes).
 - Implémentation de l'interpolation linéaire pour fluidifier le déplacement des agents entre deux nœuds.
 - Développement des premières règles de collision (rayon d'action pour manger/tuer) et gestion des bordures du monde.
- **Phase 3 : Connectivité et Visualisation (Semaine 7)**
 - Développement du serveur Socket Python (`main.py`) et du protocole de sérialisation JSON.
 - Création du client JavaFX : implémentation du thread d'écoute réseau et synchronisation avec l'Application Thread via `Platform.runLater`.
 - Rendu graphique initial : distinction visuelle Agents (Cercles) et Ressources (Carrés).
- **Phase 4 : Intelligence Artificielle Avancée (Semaines 8 à 9)**
 - Implémentation de l'algorithme A* générique dans `world.py` (heuristique Euclidienne).
 - **Spécialisation du Loup :** Création de la mémoire spatiale (`known_map`) et développement de l'algorithme `astar_memory` avec gestion du "Brouillard de Guerre".
 - Optimisation : Passage à l'heuristique de Manhattan pour les agents à recalcul fréquent.
- **Phase 5 : Équilibrage et Finalisation (Semaine 10)**
 - Calibration des constantes dans `parametres.py` (seuils de faim, coûts énergétiques du sprint).
 - Tests intensifs des scénarios d'émergence (Surpopulation vs Famine).
 - Rédaction de la documentation technique et du rapport final.

Conclusion

Le projet a permis de modéliser avec succès un micro-écosystème fonctionnel. L'architecture client-serveur (Python/Java) assure une séparation propre entre la logique et l'affichage. L'implémentation d'un `pathfinding` adapté (A* avec mémoire) et le déplacement continu constituent les réussites techniques majeures, offrant un comportement naturel et non-robotique aux agents.